Содержание

0.1 Title 3

1 Цель работы **4**

2 Задание **5**

3 Теоретическое введение **6**

4 Выполнение лабораторной работы **7**

5 Выводы **16**

Список иллюстраций

4.1 Переключение на суперпользователя 7
4.2 Установка git 7
4.3 Установка gh 7
4.4 Настройка имени и email 8
4.5 Настройка кадировки utf-8 8
4.6 Настройка имени начальной ветки 8
4.7 Настройка параметров 8
4.8 Генерация ключа 9
4.9 Генерация ключа 10
4.10 Список ключей RGP 11
4.11 Список ключей 11
4.12 Копирование ключа 11
4.13 Настройка подписи коммитов 12
4.14 Авторизация 12
4.15 Создание рабочего каталога 12
4.16 Создания репозитория 13
4.17 Клонирование репозитория 13
4.18 Переход в каталог курса 13
4.19 Удаление лишних файлов 13
4.20 Создание каталогов 14
4.21 Создание дополнительных каталогов 14
4.22 Добовление файлов в индекс 14
4.23 Создание коммита 15

Список таблиц

0.1 Title

title: «Лабораторная работа №2» subtitle: «Архитектура Компьютера» author: «Семёнов Александр Дмитриевич» —

1 Цель работы

Целью работы является изучение работы и применения средств контроля версий и освоение по работе с git.

2 Задание

Создать базовую конфигурацию для работы с git, ключ ssh, ключ pgp, подписи git, локальный каталог для выполнения и прикрепления щаданий по предмету.

3 Теоретическое введение

Системы контроля версий (Version Control System, VCS) используются для организации совместной работы коллектива над общим проектом. Как правило, освновная ветка разработки хранится в репозитории - локальном или удаленном, - к которому организован доступ всех участников. Когда разработчики вносят правки, VCS позволяет регистрировать эти изменения, объединять результаты работы разных специалистов, а при необходимости - выполнять возврат к любой из более ранних версий проекта.

В классической модели контроля версий применяется централизованный подход: все файлы хранятся в едином репозитории, а управление версиями обеспечивается выделенным сервером. Участник проекта перед началом работы с помощью специальных команд запрашивает актуальную или нужную ему версию файлов. Завершив внесение правок, он отправляет обновлённую версию обратно в хранилище. При этом все предыдущие версии сохраняются в центральном репозитории, и к ним можно обратиться в любой момент. Чтобы экономить дисковое простарнство, сервер может не сохранять полностью каждый изменённный файл, а применять дельта-компрессию - записывать только различия между последовательными верссиями.

4 Выполнение лабораторной работы

1) Для начала работы переключимся на суперпользователя при помощи команды „sudo -i“ (рис. 4.11).

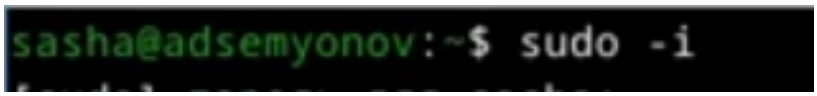


Рисунок 4.1: Переключение на суперпользователя

2) Установим git при помощи команды „dnf install git“ (рис. 4.2).

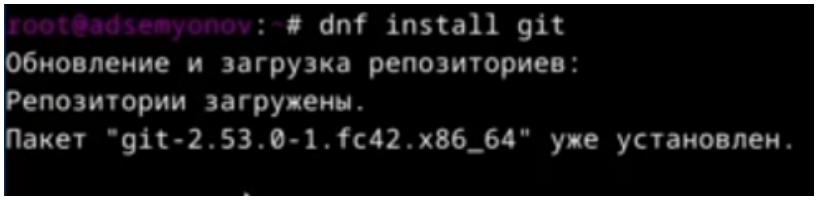


Рисунок 4.2: Установка git

3) Установим gh при помощи команды „dnf install gh“ (рис. 4.3.

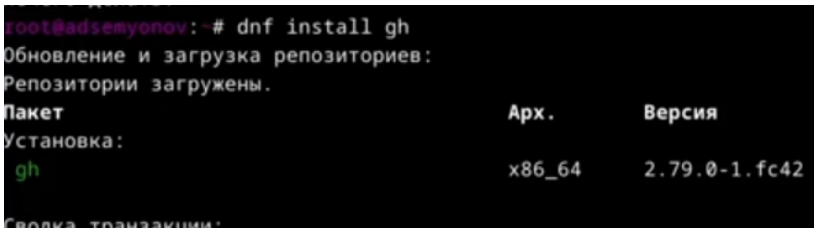


Рисунок 4.3: Установка gh

4) Зададим имя и email владельца репозитория с помощью команды „git config –global user.name“ и „git config –global user.email“ (рис. 4.4).

```
sasha@adsemyonov:~$ git config --global user.name "Alexander Semyonov"
sasha@adsemyonov:~$ git config --global user.email "1032252587@pfur.ru"
```

Рисунок 4.4: Настройка имени и email

5) Настроим utf-8 в выводе ссобщений git командной „git config –global core.quotepath false“ (рис. 4.5).

```
sasha@adsemyonov:~$ git config --global core.quotepath false
```

Рисунок 4.5: Настройка кадировки utf-8

6) Зададим имя начальной ветки (master) с помощью команды „git config –global init.defualtBranch master“ (рис. 4.6).

```
sasha@adsemyonov:~$ git config --global init.defaultBranch master
```

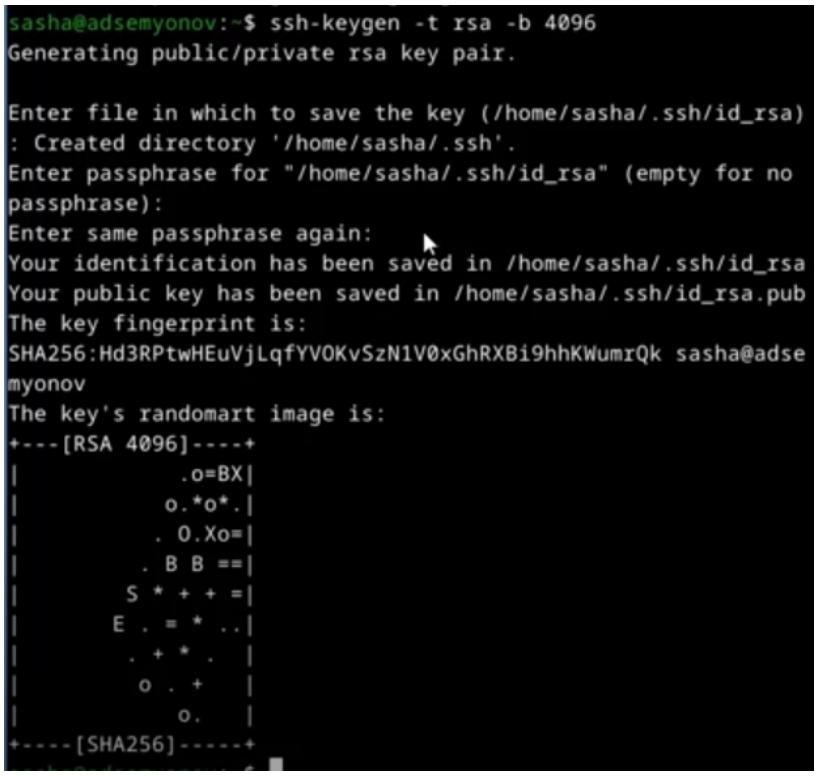
Рисунок 4.6: Настройка имени начальной ветки

7) Настроим параметры autocrlf и safecrlf для корректной работы с окончанием строк (рис. 4.7.

```
sasha@adsemyonov:~$ git config --global core.autocrlf input
sasha@adsemyonov:~$ git config --global core.safecrlf warn
```

Рисунок 4.7: Настройка параметров

8) Сгенерируем ключ ssh по алгаритму rsa с ключом размером 4096 бит (рис. 4.8).

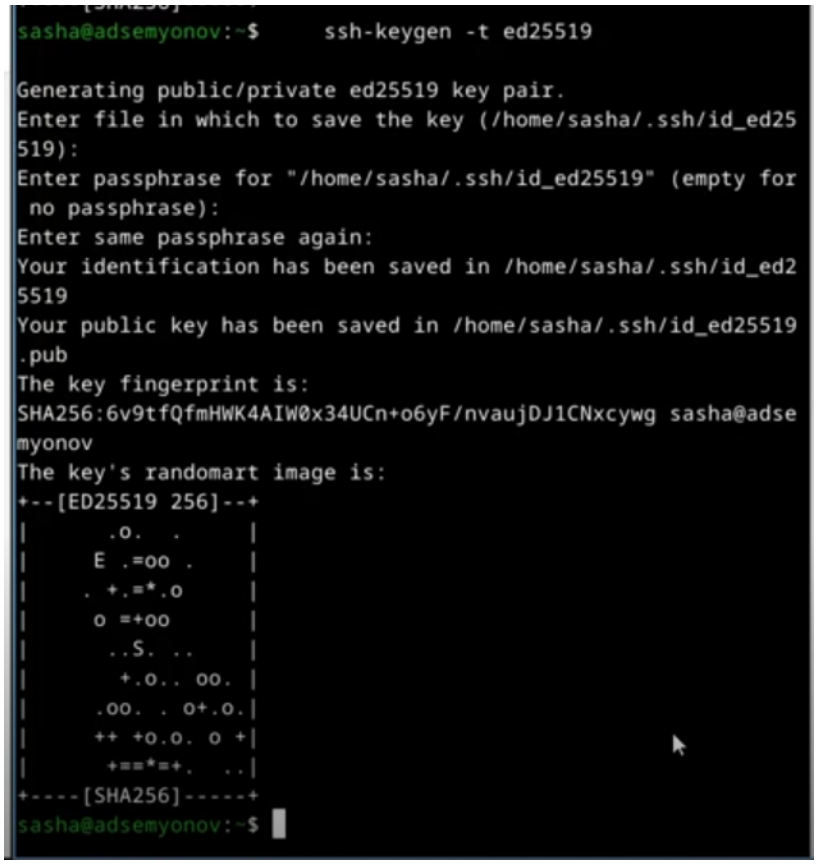


```
sasha@adsemyonov:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.

Enter file in which to save the key (/home/sasha/.ssh/id_rsa)
: Created directory '/home/sasha/.ssh'.
Enter passphrase for "/home/sasha/.ssh/id_rsa" (empty for no
passphrase):
Enter same passphrase again:
Your identification has been saved in /home/sasha/.ssh/id_rsa
Your public key has been saved in /home/sasha/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:Hd3RPtwHEuVjLqfYVOKvSzN1V0xGhRXBi9hhKWumrQk sasha@adse
myonov
The key's randomart image is:
+---[RSA 4096]----+
|            .o=BX|
|           o.*o*.|
|          . O.Xo=|
|         . B B ==|
|        S * + + =|
|       E . = * ..|
|        . + * .  |
|         o . +   |
|           o.    |
+----[SHA256]-----+
```

Рисунок 4.8: Генерация ключа

9) Сгенерируем ключ ssh по алгоритму ed25519 (рис. 4.9).



```
sasha@adsemyonov:~$     ssh-keygen -t ed25519

Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/sasha/.ssh/id_ed25
519):
Enter passphrase for "/home/sasha/.ssh/id_ed25519" (empty for
 no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/sasha/.ssh/id_ed2
5519
Your public key has been saved in /home/sasha/.ssh/id_ed25519
.pub
The key fingerprint is:
SHA256:6v9tfQfmHWK4AIW0x34UCn+o6yF/nvaujDJ1CNxcywg sasha@adse
myonov
The key's randomart image is:
+--[ED25519 256]--+
|       .o.  .     |
|      E .=oo .    |
|     . +.=*.o     |
|      o =+oo      |
|       ..S. ..    |
|        +.o.. oo. |
|      .oo. . o+.o.|
|      ++ +o.o. o +|
|       +==*=+.  ..|
+----[SHA256]-----+
sasha@adsemyonov:~$
```

Рисунок 4.9: Генерация ключа

10) Теперь сгенирируем RGP ключ для подписи коммитов (рис. 4.10).

Рисунок 4.10: Список ключей RGP

11) Выведем список ключей и скопируем отпечаток приватного ключа (**?@fig-011**.

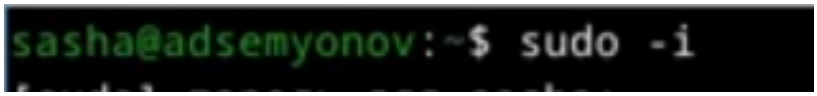


Рисунок 4.11: Список ключей

12) Скопируем сгенерированный ключ в буфер обмена (рис. 4.12).

Рисунок 4.12: Копирование ключа

13) Используя введенный email, укажем git применять его при подписки коммитов (рис. 4.13).

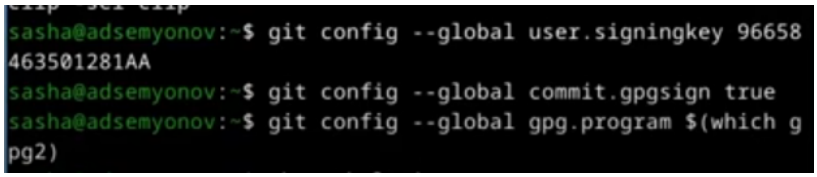


Рисунок 4.13: Настройка подписи коммитов

14) Авторизуемся на GitHub через gh (рис. 4.14).

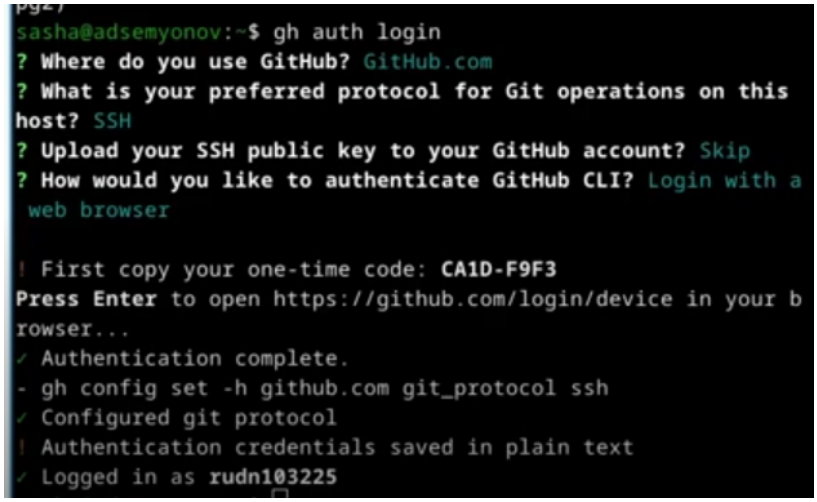


Рисунок 4.14: Авторизация

15) Создадим каталог для работы и перейдем в него (рис. 4.15).

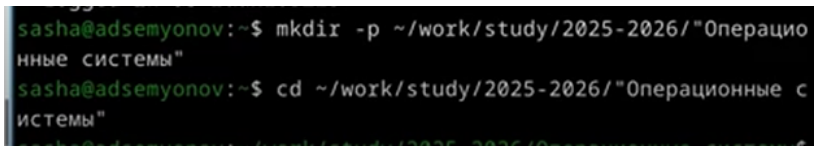


Рисунок 4.15: Создание рабочего каталога

16) Создадим репозиторий на основе шаблона (рис. 4.16).

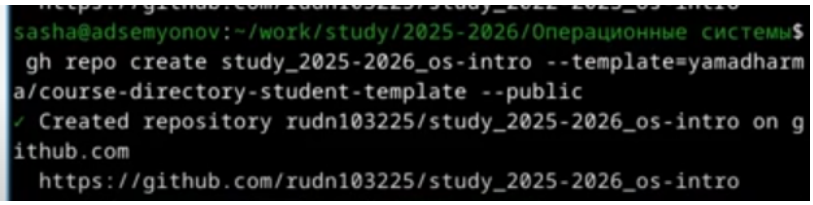


Рисунок 4.16: Создания репозитория

17) Выполним клонирование репозитория (рис. 4.17).

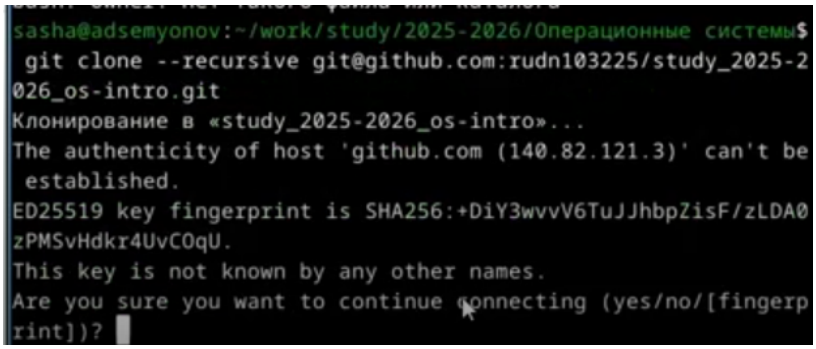


Рисунок 4.17: Клонирование репозитория

18) Перейдем в каталог курса (рис. 4.18).

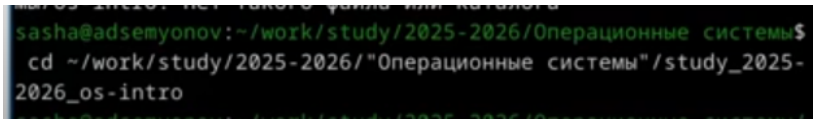


Рисунок 4.18: Переход в каталог курса

19) Удалим лишние файлы (рис. 4.19).

Рисунок 4.19: Удаление лишних файлов

20) Создадим необходимые каталоги для лабораторных работ (рис. 4.20).

```
study_2025-2026_os-intro$ echo os-intro > COURSE
sasha@adsemyonov:~/work/study/2025-2026/Операционные системы/
study_2025-2026_os-intro$ make
Usage:
  make <target>

Targets:
  list                            List of courses
  prepare                         Generate directories struct
ure
  submodule                       Update submules

sasha@adsemyonov:~/work/study/2025-2026/Операционные системы/
study_2025-2026_os-intro$
```

Рисунок 4.20: Создание каталогов

21) Продолжим создание дополнительных каталогов (рис. 4.21).

```
study_2025-2026_os-intro$ git add .
```

Рисунок 4.21: Создание дополнительных каталогов

22) Отправим созданые файлы на сервер (рис. 4.22).

```
study_2025-2026_os-intro$ git commit -am 'feat(main): make co
urse structure'
[master cb364e3] feat(main): make course structure
 2 files changed, 1 insertion(+), 14 deletions(-)
 delete mode 100644 package.json
```

Рисунок 4.22: Добовление файлов в индекс

23) Затем выполним коммит с сообщением о проделанной работе (рис. 4.23).

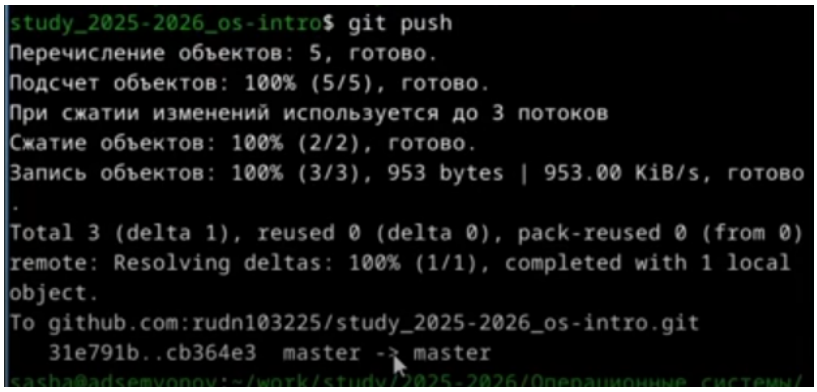


Рисунок 4.23: Создание коммита

5 Выводы

В результате выполнения лабораторной работыя приобрел навыки, необходимые для работы с git, настроил каталоги курса для дальнейшей работы, создал ssh и gpg ключи и авторизовался в gh.